

DS5899: Special Topics in Data Science

Leveraging NLP in Asset Management

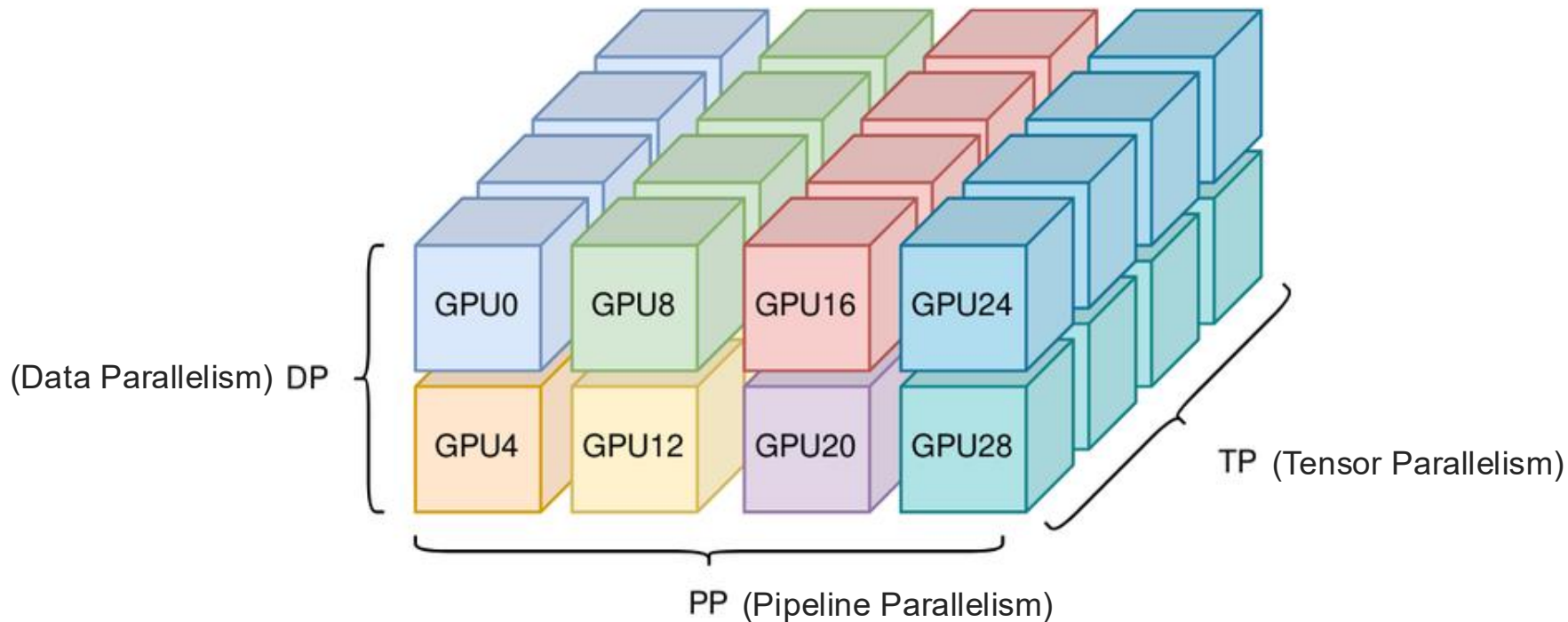
Week 5: Infrastructure and Optimization

Instructor: Che Guan, Ph.D.

Part 1:

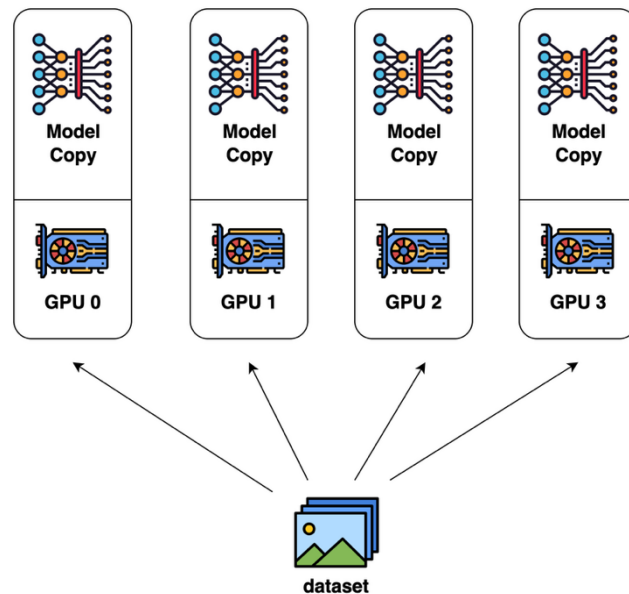
Parallelism

LLM's Training in Parallelism



Data Parallelism (DP)

- In DP training, the dataset is split into several shards, each shard is allocated to a GPU device.
- Each device will hold a full copy of the model replica and trains on the dataset shard allocated.
- After back-propagation, the gradients of the model will be all-reduced so that the model parameters on different devices can stay synchronized.
- As each GPU holds a copy of the whole model weights, this brings redundancy.

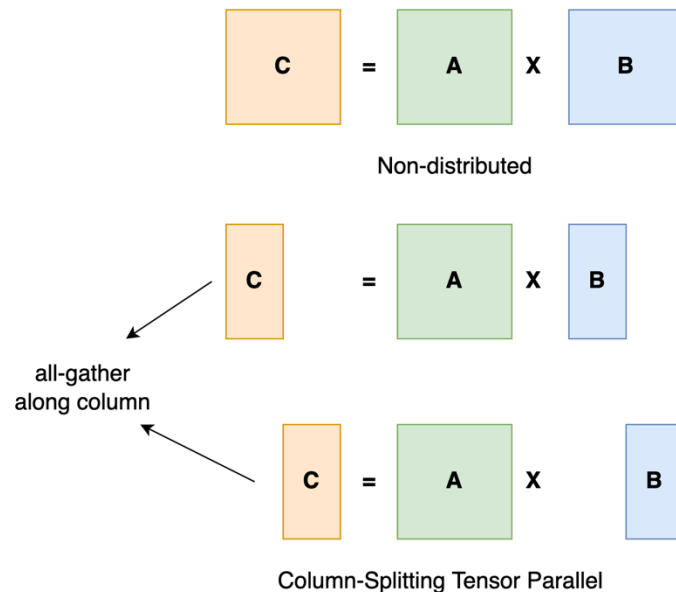


Model Parallelism (MP)

- Having a copy of the model on every GPU device brings about redundancy. Therefore, another parallelism mechanism is model-wise, i.e, the model is split across devices:
 - **Tensor Parallelism (TP):** conduct parallel computing within an operation, such as Matrix Multiplication. It split within the transformer layer.
 - **Pipeline Parallelism (PP):** conduct parallel computing across layers — split the transformer layers.

Tensor Parallelism(TP)

- TP training is to split a tensor into N (in the example) chunks along a specific dimension and each GPU device only holds 1/N of the whole tensor while not affecting the correctness of the computation graph
- This requires additional communication to make sure that the result is correct.



Sequence Parallelism(SP)

- SP extends TP by distributing computing load and activation memory across multiple GPUs along the sequence dimension of transformer layers.

Idea: Layernorm and dropout activations can be parallelized across the sequence dimension, so we distribute those across tensor parallel ranks.

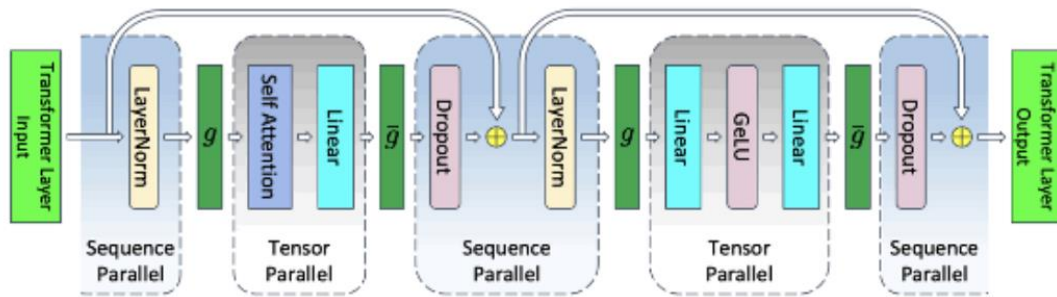
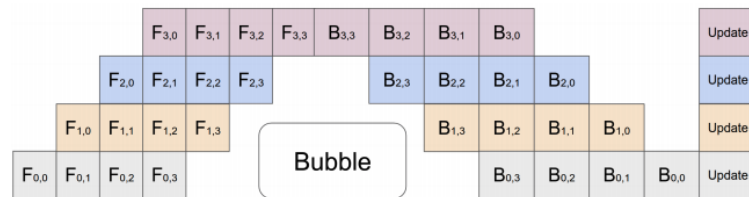
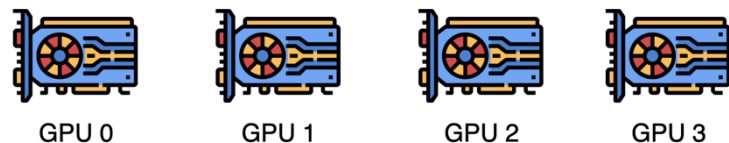
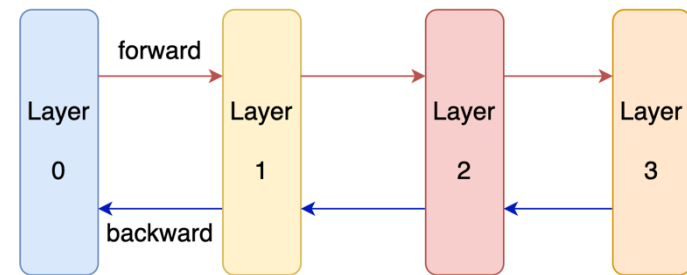


Figure 5: Transformer layer with tensor and sequence parallelism. g and $\bar{g}$ are conjugate. g is all-gather in the forward pass and reduce-scatter in the backward pass. $\bar{g}$ is reduce-scatter in forward pass and all-gather in backward pass.

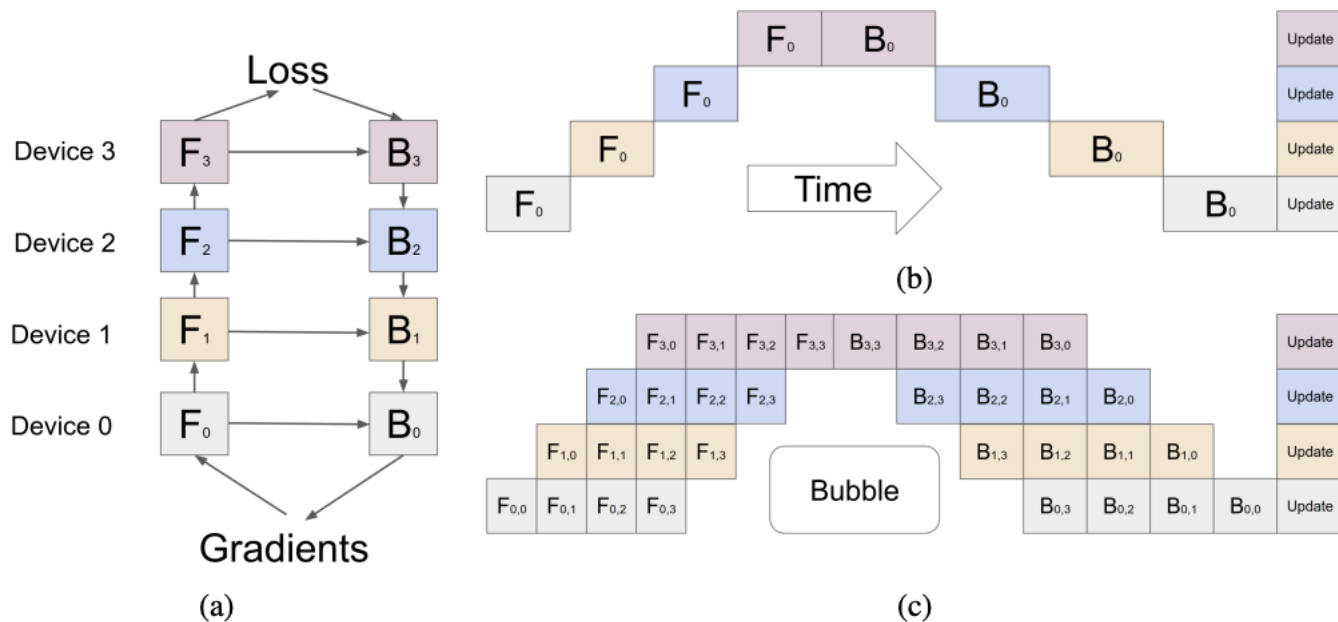
Pipeline Parallelism (PP)

- PP splits the model layer into several chunks, each chunk is given to a GPU device.
- During the forward pass, each device passes the intermediate activation to the next stage.
- During the backward pass, each device passes the gradient of the input tensor back to the previous pipeline stage.
- One drawback of PP training is that there will be some bubble time where some devices are engaged in computation, leading to waste of computational resources.



Pipeline Parallelism - Gpipe

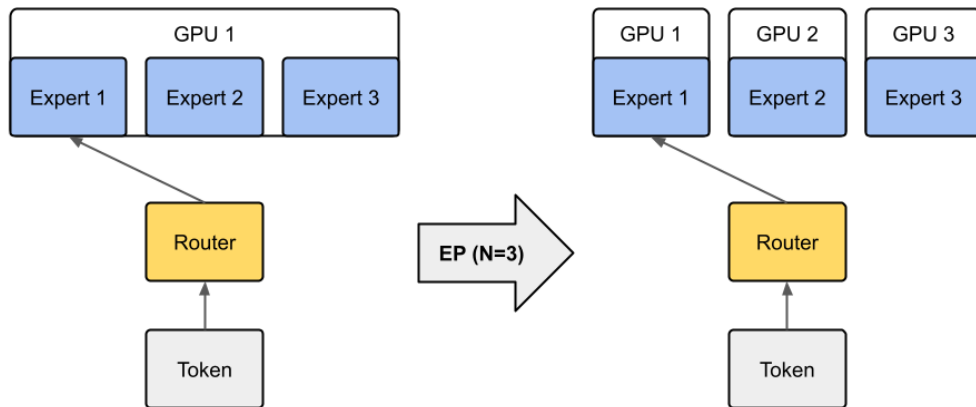
- The naive model parallelism strategy leads to severe underutilization due to the sequential nature of the network: only one accelerator is active at a time.
- GPipe divides the input mini-batch into smaller micro-batches, enabling different accelerators to work on separate micro-batches at the same time.



Expert Parallelism (EP)

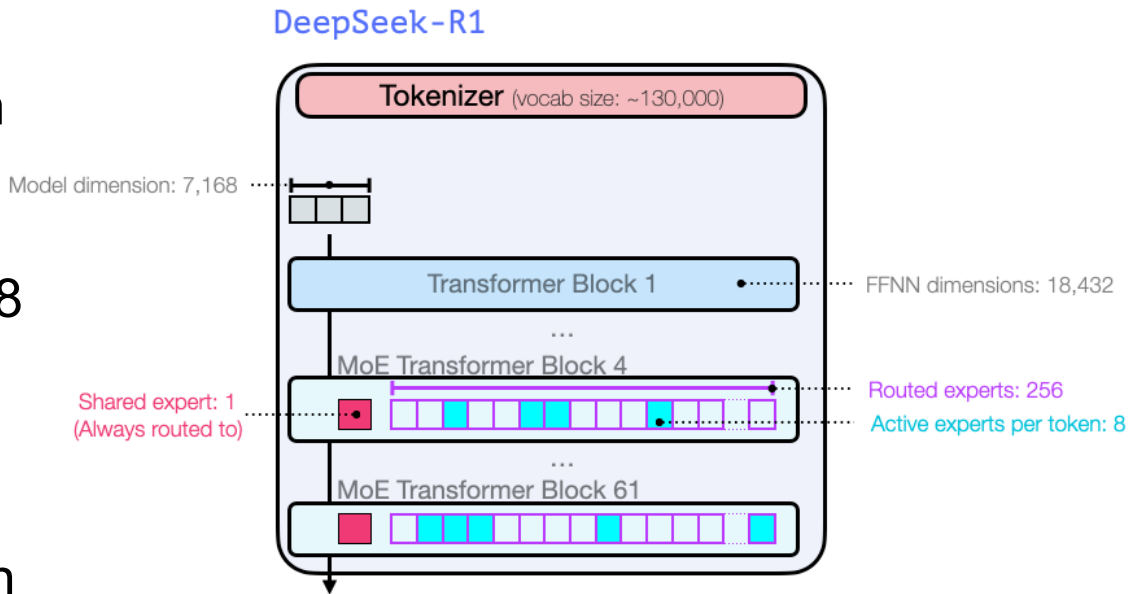
- EP distributes experts of an MoE across GPUs. EP is applied to only the expert layers thus does not impact the parallel mapping of the rest of layers.

Expert Parallelism applied on Mixture-of-Experts.



DeepSeek-R1 Technical Specifications

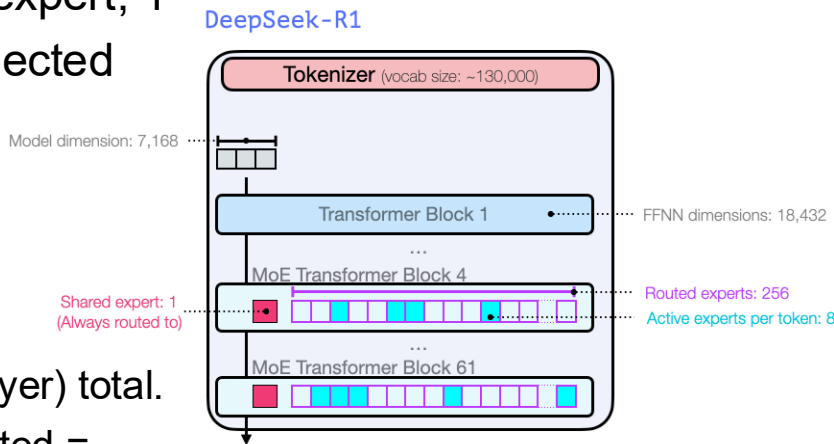
- **Layer Architecture:** Total of 61 layers. The Attention layers are standard; the first 3 FFN layers are standard; the remaining 58 FFN layers are replaced by MoE.
- **Parameter Count:** Total parameters are **671B**, with **37B** activated per token.



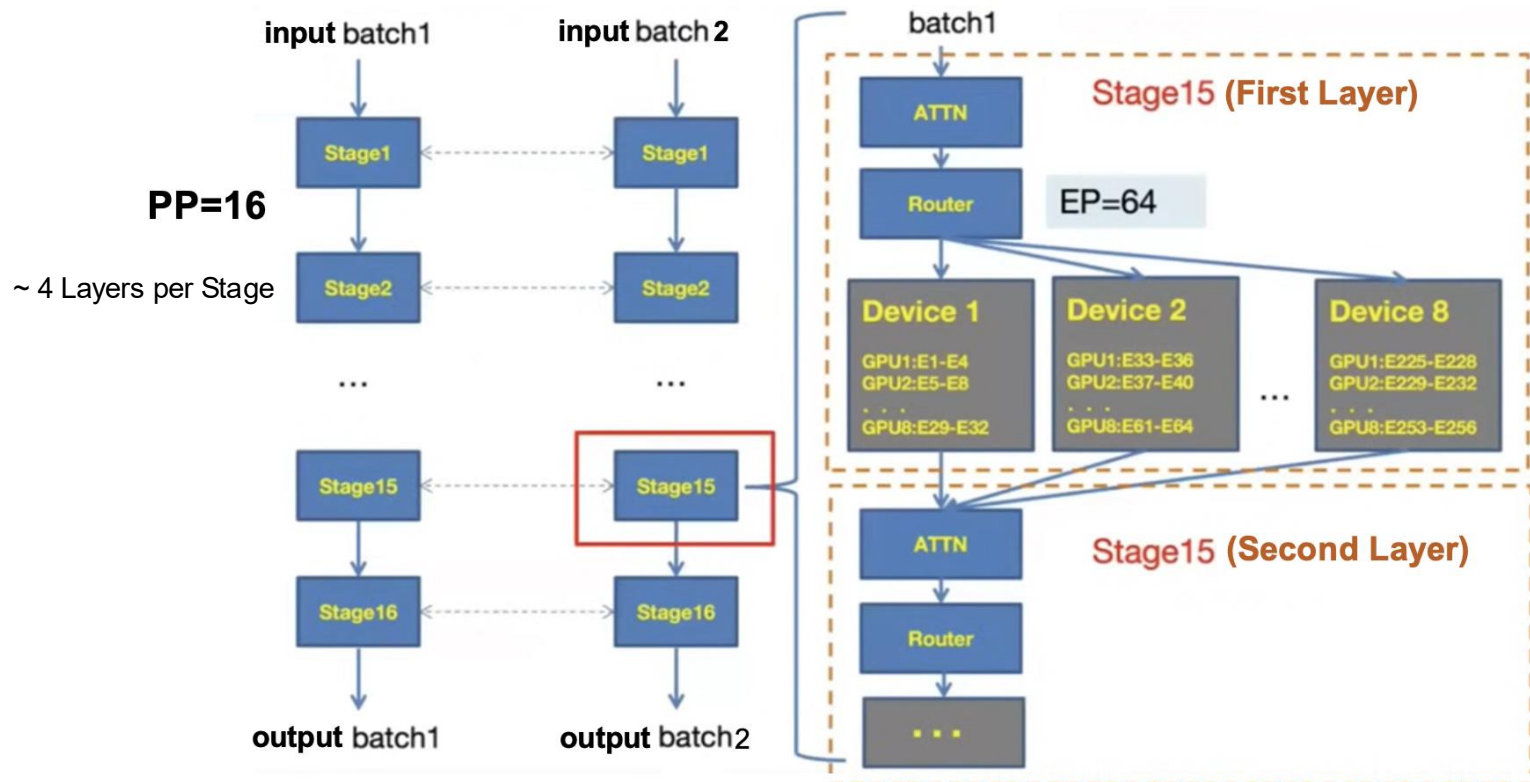
[Image Resource: The Illustrated DeepSeek-R1](#)

Technical Specifications

- **Expert Activation:** For each layer, 1 shared expert, 1 router, and 8 routed experts are activated (selected from a pool of 256 routed experts).
- **Expert Composition:**
 - **Routers:** 1 per layer, 58 total.
 - **Shared Experts:** 1 per layer, 58 total.
 - **Routed Experts:** 14,848 (256 per layer x 58 layer) total.
 - **Grand Total Experts:** 58 Shared + 14,848 Routed = 14,906 total experts.
 - **Total Activated Experts:** (1 Shared + 8 Routed) x 58 layers = 522 activated experts.
- **Memory Estimate:** A single routed expert has 44M parameters. If stored in BF16, it requires approximately 84MB of VRAM.



Infrastructures - Training Framework

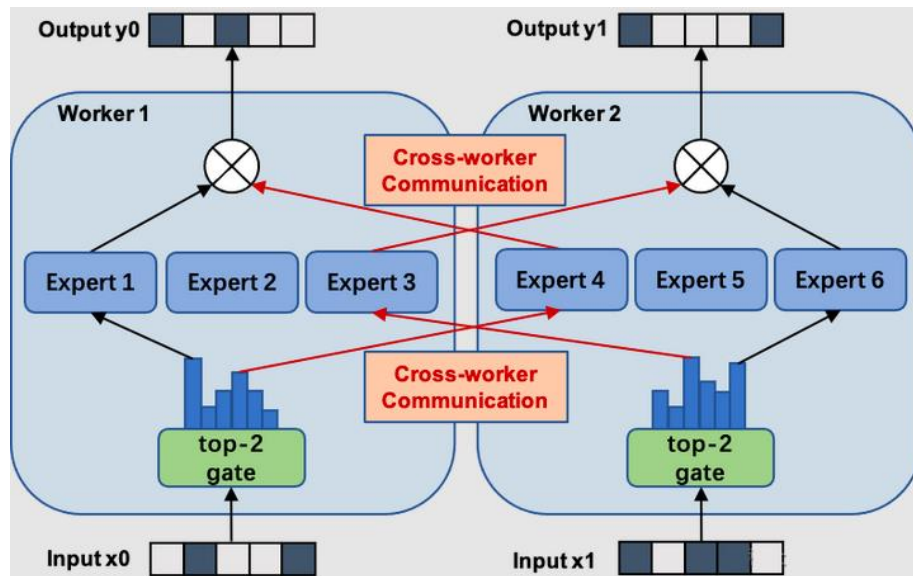


Training Framework, *Continued*

- DeepSeek-V3 employs **64-way EP** using 8 nodes, each equipped with 8 GPUs and ZeRO-1 DP
- Additionally, DeepSeek-V3 implements **16-way PP**. To enhance efficiency in PP, the team developed the **DualPipe** algorithm, which overlaps communication and computation phases during both forward and backward passes, effectively reducing pipeline bubbles.

An Example of Sparse MoE with Two-way EP

- A **sparse MoE** architecture reduces training costs for LLMs by activating only a subset of smaller experts during each iteration, using a trainable gating mechanism to optimize computation.
- In the example, an LLM with 6 experts is trained using **Two-way EP**:
 - Experts 1–3 are placed on the 1st GPU,
 - Experts 4–6 are placed on the 2nd GPU.



Infrastructures- Compute Clusters

- DeepSeek-V3 is trained on a cluster equipped with **2,048 NVIDIA H800 GPUs**. Each node in the H800 cluster contains 8 GPUs connected by NVLink and NVSwitch within nodes. Across different nodes, InfiniBand interconnects are utilized to facilitate communications.
- A single pipeline stage contains 128 GPUs (2,048 GPUs / 16 PP stages). Within each pipeline stage, 64-way Expert Parallelism is used.

Parallelism for Optimizers

- During model training, the GPU needs to store several types of data.
- ZeRO is designed with 3 optimization stages, each progressively sharding different parts of the model state:
 - **ZeRO-1:** Shards the optimizer states (Optimizer States Sharding).
 - ZeRO-2: Shards both optimizer states and gradients (Optimizer States & Gradients Sharding).
 - ZeRO-3: Shards the optimizer states, gradients, and model parameters (Optimizer States & Gradients & Parameters Sharding).



GShard

GShard introduced the concept of sharding feed-forward (FF) experts across multiple devices, so that each device:

- Processes a partition of tokens (a token “group”), computing gating scores (often with softmax) to decide which experts these tokens should go to.
- Dispatches embeddings to remote devices that host the relevant experts.
- Combines the outputs from those experts, which are sent back from other devices, before proceeding to the next layer in the Transformer.

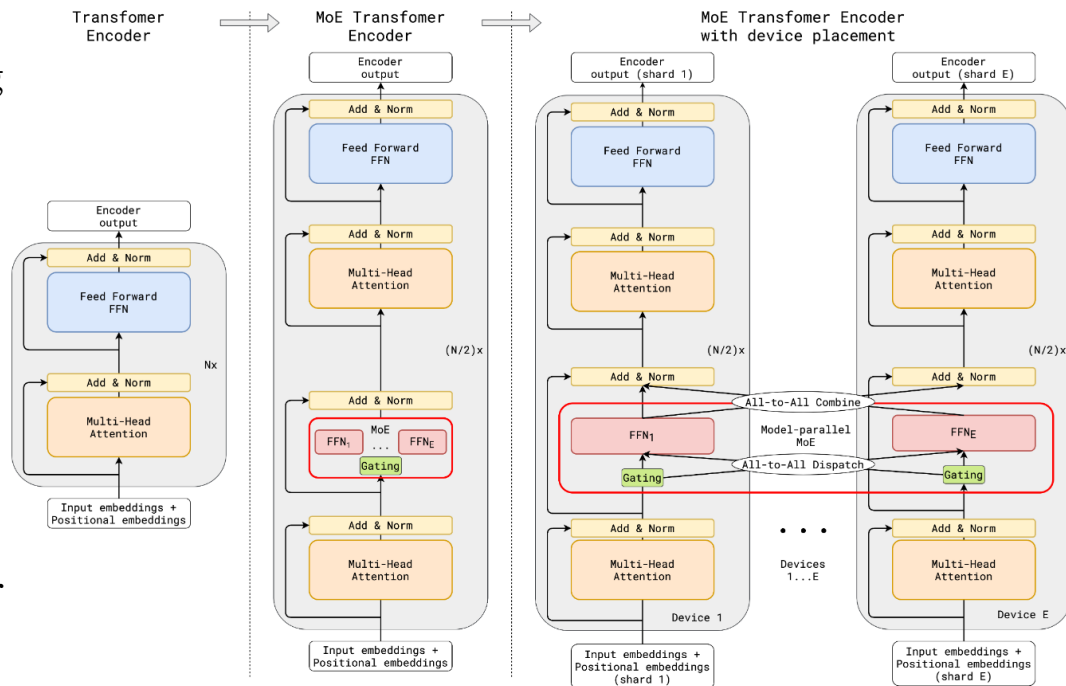
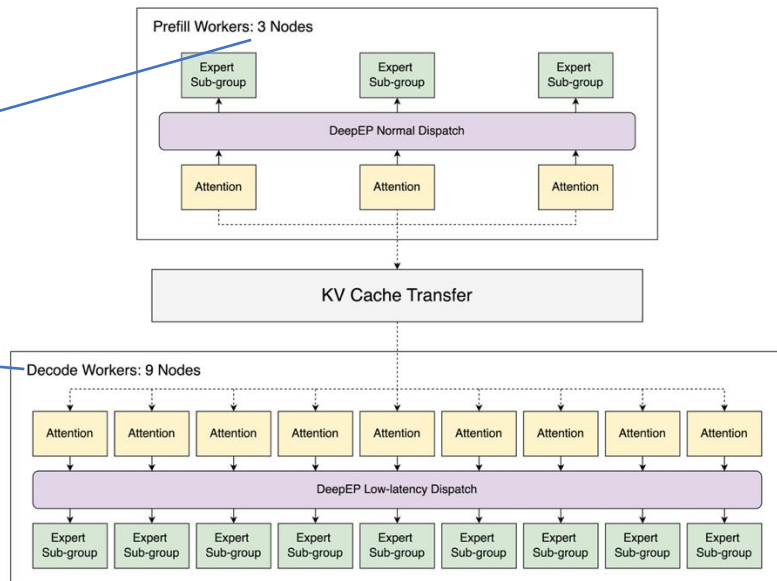


Figure 2: Illustration of scaling of MoE Transformer Encoder Layers. Decoder modification is similar. (a) Standard Transformer. (b) Replacing every other feed forward layer with a MoE layer (c) The MoE layer is sharded across multiple devices, while all other layers are replicated.

Image Source: Lepikhin et al., [GSHARD](#)

Deploying DeepSeek with PD Disaggregation and Large-Scale EP on 96 H100 GPUs

- DeepSeek is a popular open-source LLM praised for its strong performance (DeepSeek-V3 is trained with **2,048 GPUs**). However, its large size and unique architecture, which uses MLA and MoE, require an advanced system for efficient serving at scale.
- LMSYS runs on **12** nodes in the Atlas Cloud, each equipped with **8** H100 GPUs. It uses PD disaggregation and large-scale EP. This represents the first open-source implementation to nearly match the throughput reported in the official DeepSeek blog at large scale.



DualPipe and Computation-Communication Overlap

Once the **MLP(B)** (backward for input) computation finishes, two things happen simultaneously:

Communication: The system starts **DISPATCH(B)** to send the calculated input gradients to the appropriate experts/nodes.

Computation: The GPU immediately begins **MLP(W)** (backward for weights) as this calculation relies on local data and doesn't need to wait for the dispatch to finish

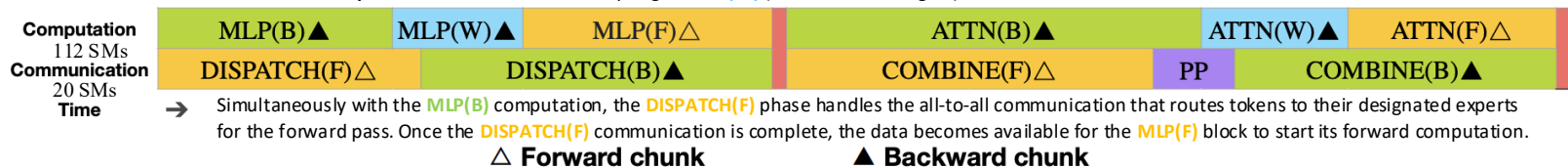
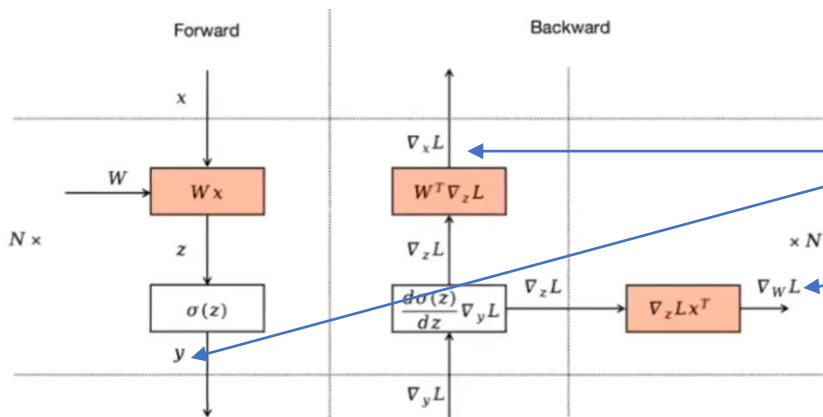


Figure. Orange denotes forward, green denotes "backward for input", blue denotes "backward for weights", purple denotes Pipeline Parallelism (PP) communication, and red denotes barriers. Both all-to-all and PP communication can be fully hidden.

Computation Tasks (Executed on 112 SMs):

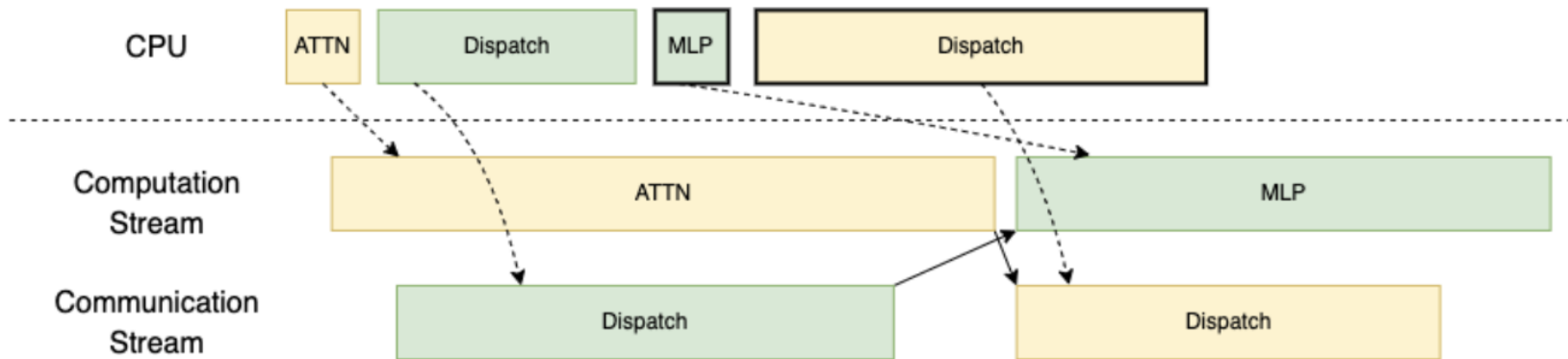
1. **Forward (F):** Calculate $y = \sigma(z) = \sigma(W \cdot x)$, labeled as MLP(F)
2. **Backward (B):** Calculate $\nabla_x L$ (Gradient of Loss with respect to input x , labeled as MLP(B))
3. **Weight (W):** Calculate $\nabla_w L$ (Gradient of Loss with respect to weights W , labeled as MLP(W))

Communication Tasks: Handle the gradient of Loss with respect to x ($\nabla_x L$), executed on 20 SMs.



LMSYS: 2-batch Overlap with a Proper Launch Order

- A "proper" launch order is a choreographed sequence that ensures GPU's compute units and its communication hardware (like NICs/NVLink) are active at the same time.
 - **Perfect Synchronization:** The system starts the communication for the next micro-batch (Batch 1) immediately, so it runs in the background while the GPU is still doing the math for the current micro-batch (Batch 0).
 - **Hiding Latency:** By the time the GPU finishes the math for Batch 0, the data for Batch 1 has already arrived over the network. This "hides" the communication time behind the computation time.



DualPipe Scheduling for 8 PP Ranks and 20 Micro-batches in Two Directions



Pipeline Parallelism (PP) = 8, a schedule across 8 Devices (0 – 7) over time;
20 minibatches total, 10 each for forward and reverse directions (0 → 7, 7 → 0).

- The micro-batches in the reverse direction are symmetric to those in the forward direction; Two cells enclosed by a shared black border exhibit overlapping computation and communication.
 - Minimal GPU Bubbles:** Achieved through a global load-balancing scheduling strategy and a computation/communication overlap strategy for individual cards.
 - Dual-Batch Processing:** Each device can process two batches simultaneously—one forward and one backward—where one handles computation while the other handles communication.
 - Granularity Control:** During backward propagation for each batch, input gradients are communicated independently forward, while local weight updates are calculated independently. This is managed through precise granularity control.

Part 2:

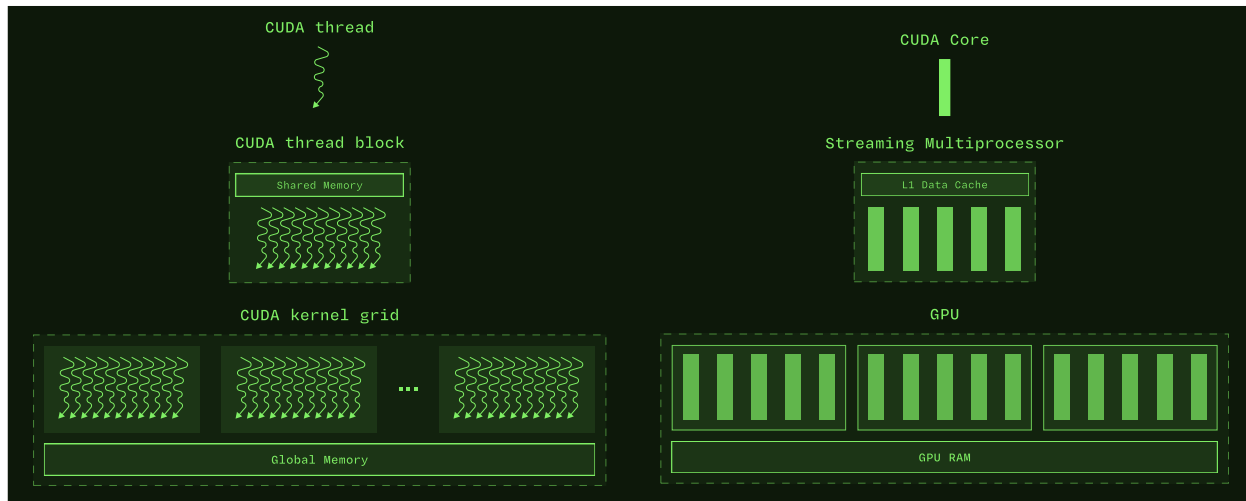
Quantization

CUDA: Streaming Multiprocessor (SM)

- Each GPU architecture includes several Streaming Multiprocessors (SMs), which are general-purpose processors designed with low clock rates and small caches.
- Their primary function is to execute multiple thread blocks in parallel, taking on the next block as soon as one completes.
- While SMs support instruction-level parallelism, they do not feature branch prediction.

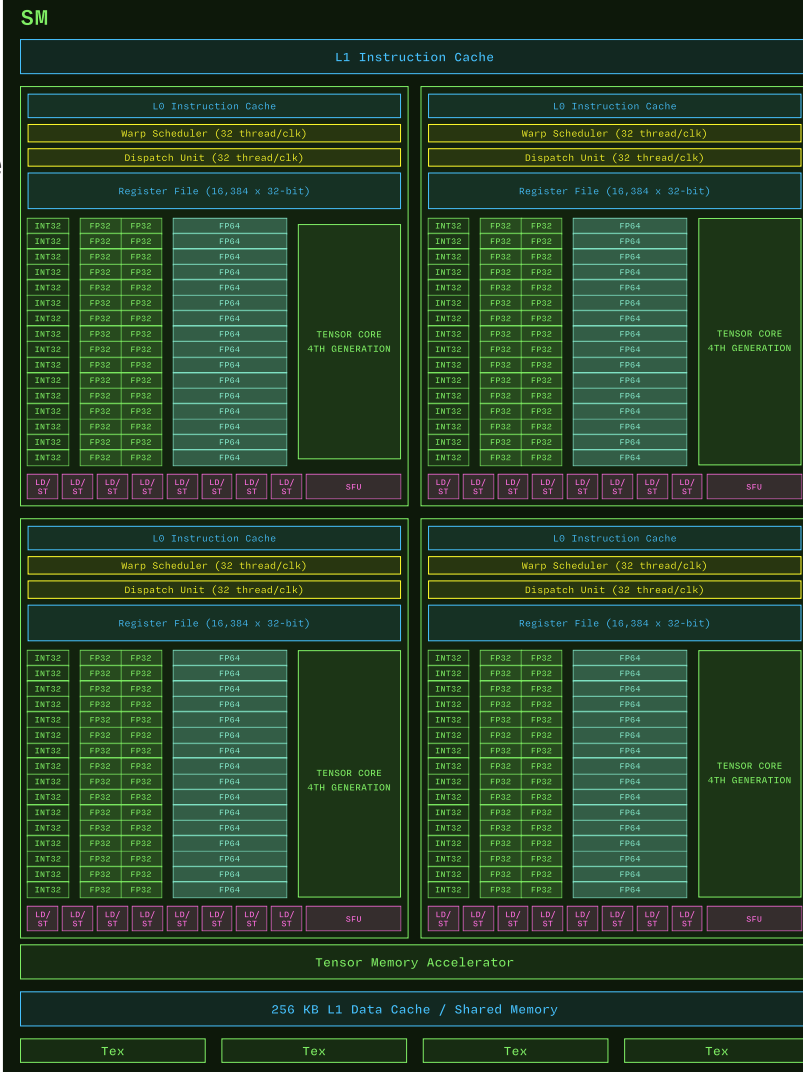
CUDA Thread and WARP

- Hardware groups threads that execute the same instruction into *warps*
 - A warp is a group of threads that are scheduled together and execute in parallel
- All threads in a warp are scheduled onto a single SM. A single SM typically executes multiple warps, at the very least all warps from the same thread block.
- Several SM constitute the whole GPU unit (which executes the whole Kernel Grid).



Streaming Multiprocessor (SM)

- **SMs** both execute computations and store state available for computation in registers w/ associated cache.
- **Special Function Units (SFUs)** accelerate certain arithmetic operations.
- **Load/Store Units (LSUs)** dispatch requests to load or store data to the memory subsystems of the GPU
- **The Warp Scheduler** of the SM decides which group of threads to execute on each clock cycle.
- **CUDA Cores** are GPU cores that execute scalar arithmetic instructions.
- **Tensor Cores** are GPU cores that operate on entire matrices with each instruction.
- **Tensor Memory Accelerators** are specialized hardware in Hopper & Blackwell architecture used to accelerate access to multi-dimensional arrays in GPU RAM.
- **The register file** of the SM is the primary store of bits in between their manipulation by the cores.
- **The L1 data cache** is the private memory of the SMs.

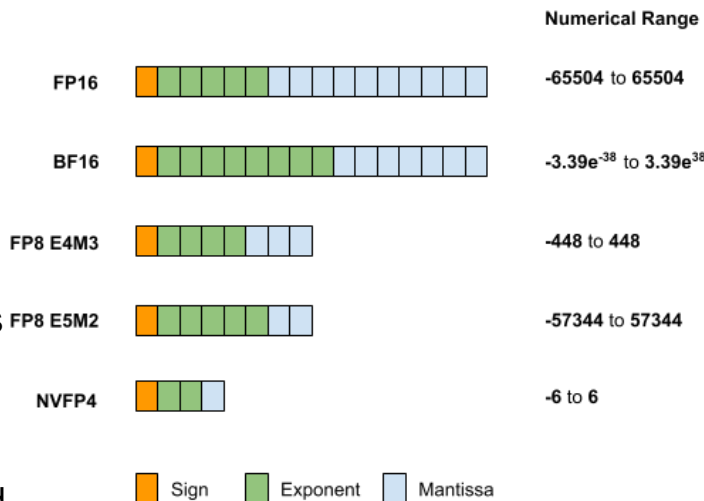


Quantization Data Types

- Data types like FP32, FP16, BF16, and FP8 affect computational resource requirements, impacting model speed and efficiency. Floating-point numbers use bits divided into three components:
 - **Sign:** A single bit that indicates positivity (0) or negativity (1).
 - **Exponent:** Encodes the power of the base (usually 2) and defines the datatype's range.
 - **Significand/Mantissa:** Represents significant digits; the length influences numerical precision.
- The formula used for this representation is: $x = (-1)^{\text{sign}} \times 2^{\text{exponent}} \times \text{mantissa}$
 - For FP8, you might find E4M3 indicating that 4 bits are used for the exponent and 3 bits for the mantissa.

Three Key Elements Can Be Quantized

- **Model Weights:** Reducing memory footprint by quantizing weights lowers storage needs,
 - e.g., from approximately 14 GB (7 billion parameters $\times$ 2 bytes per parameter) in FP16/BF16 format to about 7 GB in FP8 format
- **Model Activations:** Quantizing intermediate outputs can enhance inference speed by using specialized tensor core.
- **KV Cache:** Relevant for decoder models, this cache increases memory usage based on sequence length and model layers, aiding in autoregressive token generation.
 - The size of the KV cache is influenced by the sequence length, and the number of layers and heads. For the Llama2 7B model, which has a long context window of 4096 tokens, the KV cache can add several gigabytes to the overall memory footprint.



Quantization Algorithms: Affine Quantization

$$x_q = \text{clip} \left(\text{round} \left(\frac{x}{s} + z \right), \alpha_q, \beta_q \right)$$

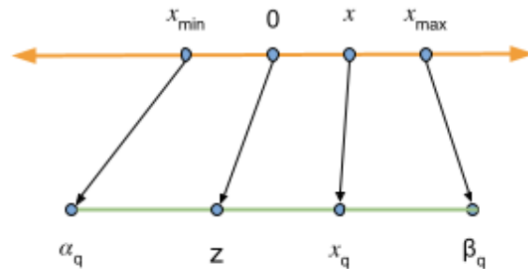
- x : The original high-precision value.
- x_q : The resulting quantized value.
- s : The **scale factor**, a positive real number used to map the original range to the new representation.
- z : The **zero point**, which ensures that the real-value zero is exactly representable in the quantized space.
- round : Converts the scaled value to the nearest integer representation.
- clip : Uses α_q and β_q to ensure values do not exceed the boundaries of the target format (e.g., -128 to 127 for INT8).

Dequantization (Recovery)

To use the quantized weights during a forward pass or for inference, we approximate the original value (x') using:

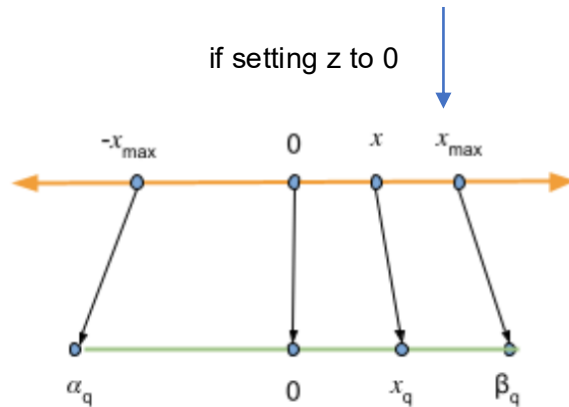
$$x \approx x' = s \cdot (x_q - z)$$

Where x' is the rounded approximation of the original value.



Affine/Asymmetric Quantization

if setting z to 0



Symmetric Quantization

Quantization Algorithms: AbsMax

- The AbsMax (Absolute Maximum) Quantization algorithm is a symmetric quantization technique, meaning the range is centered around zero.

3. Calculating the Scale Factor (s)

The scale factor s is determined by the maximum and minimum values of the original data relative to the target range:

$$s = \frac{\max(|x_{\max}|, |x_{\min}|) \cdot 2}{\beta_q - \alpha_q}$$

4. Example: FP16 to FP8 Symmetric Quantization

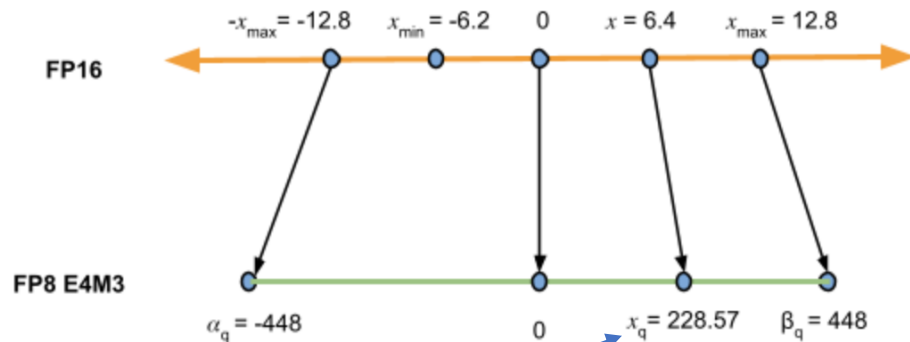
Using the **AbsMax algorithm**, the process is demonstrated as follows:

- Determining the Scale (s):**

$$s = \frac{\max(|12.8|, |-6.2|) \cdot 2}{448 - (-448)} = 0.028$$

- Calculating the Quantized Value (x_q):** Given an original value $x = 6.4$:

$$x_q = \text{clip} \left(\text{round} \left(\frac{6.4}{0.028} \right), -448, 448 \right) = 228.57$$



Quantization Granularity

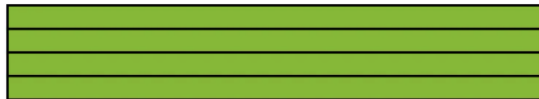
- Per-tensor (or per-layer) quantization:
 - All values within the tensor are quantized using the same set of quantization parameters.
- Per-channel quantization:
 - Different quantization parameters are used for each channel (typically along the channel dimension in convolutions).
- Per-block (or per-group) quantization:
 - This provides more fine-grained control by dividing the tensor into smaller blocks or groups, each with its own quantization parameters.

Per-Tensor:



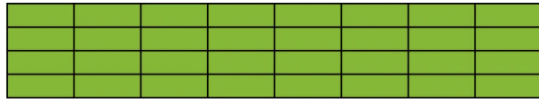
All elements share the same scale and zero-point

Per-Channel:



Each channel (horizontal strip) has its own scale and zero-point

Per-Group:



Each group (small block) has its own scale and zero-point

Quantization Approaches

- Post-Training Quantization (PTQ)
 - Weight-Only Quantization: Involves quantizing only the trained model's weights, requiring no additional data.
 - Weights and Activations Quantization: Utilizes representative input data to collect activation statistics for calibration, determining scaling and zero-point values.
 - Static Quantization: Uses a fixed calibration dataset to compute quantization parameters once.
 - Dynamic Quantization: Computes quantization parameters during inference, allowing them to vary for each input without needing a calibration dataset.
- Quantization Aware Training (QAT):
 - By simulating low-precision arithmetic, QAT allows model parameters to adapt to quantization errors. It uses "fake quantization" modules during training while maintaining high-precision computations for gradient updates.

Detailed Performance Parameters of NVIDIA GB200 NVL72 and GB200 AI Chip

	GB200 NVL72	GB200 Grace Blackwell Superchip
Configuration	36 Grace CPU : 72 Blackwell GPUs	1 Grace CPU : 2 Blackwell GPU
FP4 Tensor Core²	1,440 PFLOPS	40 PFLOPS
FP8/FP6 Tensor Core²	720 PFLOPS	20 PFLOPS
INT8 Tensor Core²	720 POPS	20 POPS
FP16/BF16 Tensor Core²	360 PFLOPS	10 PFLOPS
TF32 Tensor Core²	180 PFLOPS	5 PFLOPS
FP64 Tensor Core	3,240 TFLOPS	90 TFLOPS
GPU Memory Bandwidth	Up to 13.5 TB HBM3e 576 TB/s	Up to 384 GB HBM3e 16 TB/s
NVLink Bandwidth	130TB/s	3.6TB/s
CPU Core Count	2,592 Arm® Neoverse V2 cores	72 Arm Neoverse V2 cores
CPU Memory Bandwidth	Up to 17 TB LPDDR5X Up to 18.4 TB/s	Up to 480GB LPDDR5X Up to 512 GB/s

DeepSeek FP8 Training

- To address the sensitivity of low-precision formats like FP8 to outliers, DeepSeek performed granular partitions:

- Input:** The input tensor is partitioned into tiles where each individual tile has a dimension of 1×128 . One scaling factor is calculated and applied per 128 elements along that row segment.
- Weight:** The weight matrix is partitioned into blocks where each individual block has a dimension of 128×128 . This means a single scaling factor is shared across a 128×128 block of weights.

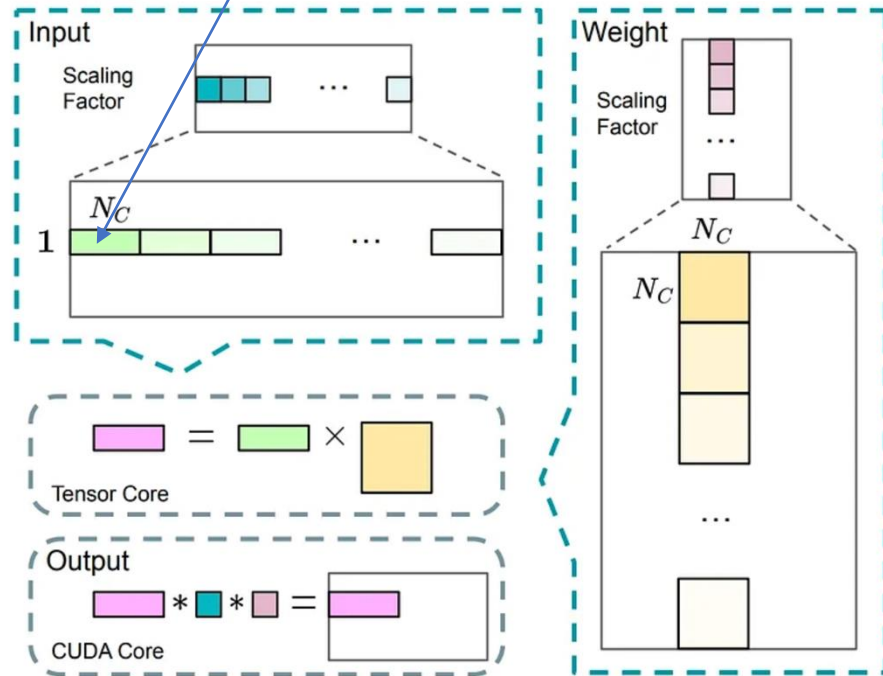
- The Loop:** After the Tensor Core performs the FP8 matrix multiplication, the results are returned to the CUDA Core. The CUDA Core performs Dequantization by multiplying by the two scaling factors (one from input, one from weight) to convert the result back to FP32 for the next operation.

- Extra Storage:** These scaling factors are stored separately to ensure the original data can be recovered accurately during the dequantization step.

Visualizing the 1×128 Tile

If your model has a hidden dimension of, for example, $d_{model} = 4096$, the "long dimension" for a single token would be divided as follows:

- Total Elements:** 4096.
- Tile Size:** 128 elements.
- Number of Tiles:** $\frac{4096}{128} = 32$ tiles per token.



WGMMMA (Warp Group Matrix Multiply-Accumulate) & Hybrid Accumulation Precision

A primary challenge on the H800 GPU is that **Tensor Core** FP8 accumulation precision is limited to approximately **14 bits**, which is significantly lower than full FP32.

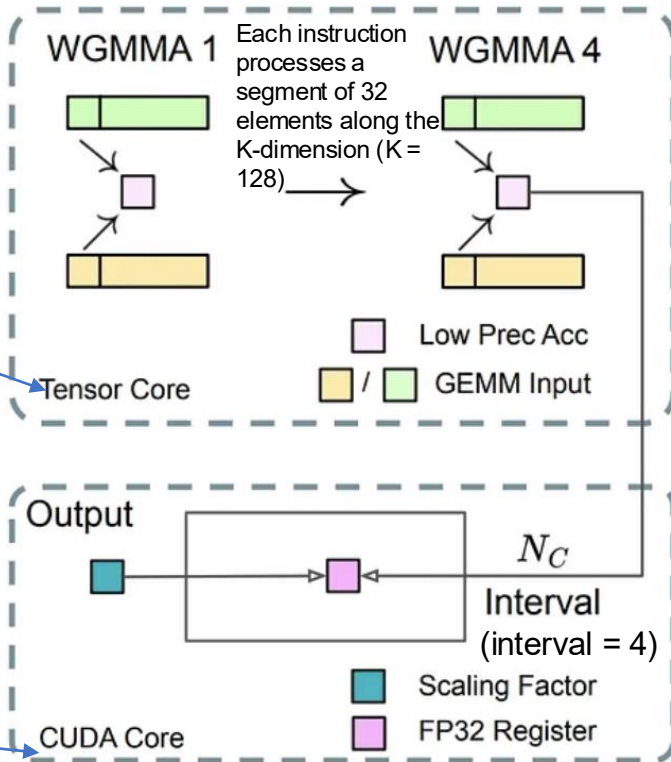
Increasing Accumulation Precision

DeepSeek overcomes this by decomposing large matrix multiplications into fine-grained segments along the K -dimension. For a multiplication $C_{64 \times 128} = A_{64 \times 128} \times B_{128 \times 128}$:

- **Decomposition:** $A = [a1, a2, a3, a4]$ and $B = [b1, b2, b3, b4]^T$.
- **Partial Sums:** $C = a1 \cdot b1 + a2 \cdot b2 + a3 \cdot b3 + a4 \cdot b4$.

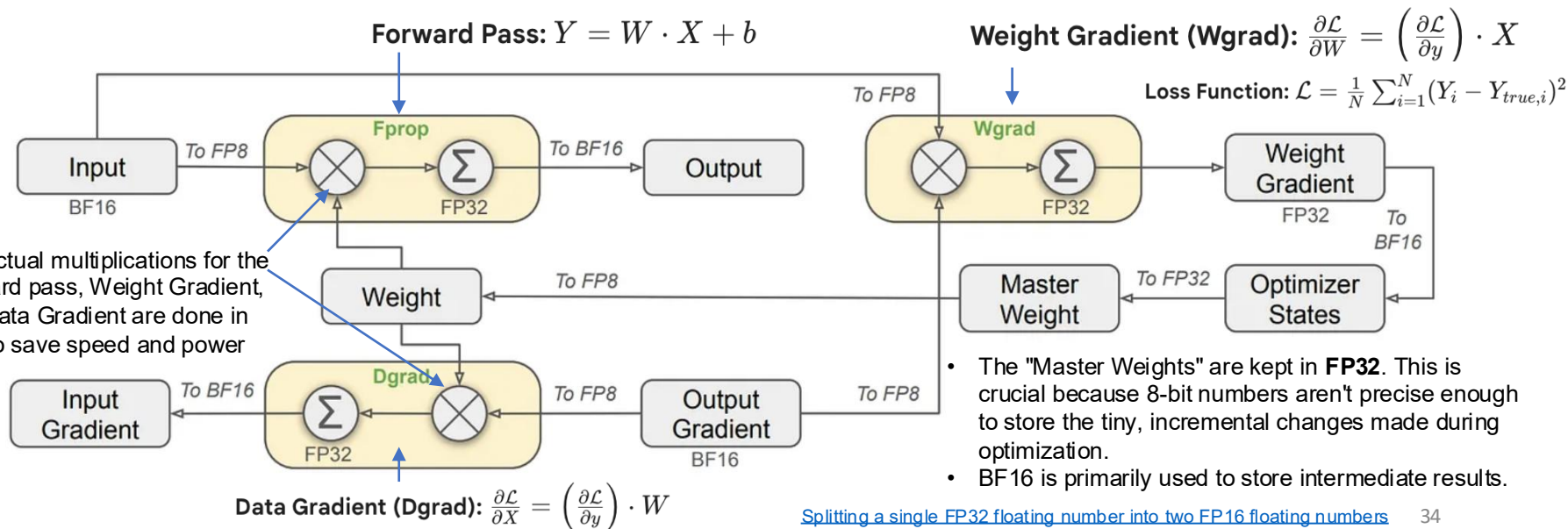
Hardware Synergy

1. **Tensor Core Phase:** Performs low-precision MMA operations.
2. **Interval Transfer (N_c):** After reaching a specific interval of WGMMAs (e.g., every 4 intervals), partial results are moved to **FP32 registers** on the **CUDA Cores**.
3. **CUDA Core Phase:** Final accumulation is performed in full FP32, preventing overflow and ensuring training accuracy.

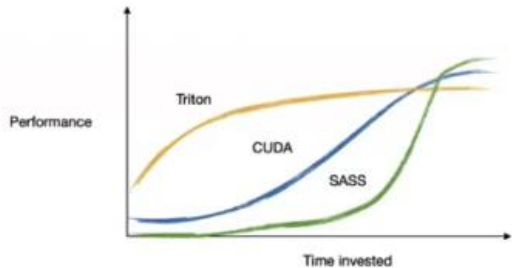


Mixed Precision Framework

- Training LLMs with FP8 is non-trivial. The challenges stem from issues such as data underflow or overflow, coupled with quantization errors arising from the narrower dynamic range and reduced precision inherent in FP8 data formats. These challenges cause numerical instabilities and irreversible divergences throughout the training process.



Why Triton (Seemingly) Succeeds



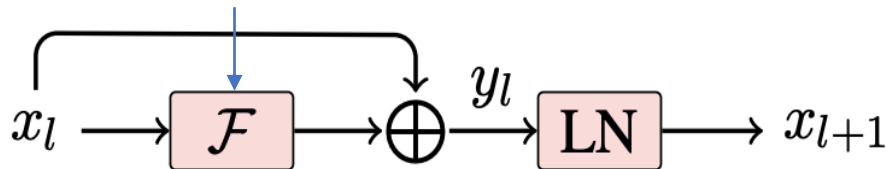
Feature	Triton	CUDA (C++/PTX)	SASS (Streaming Assembly)
Abstraction Level	High (Python DSL)	Low to Mid (C++/Assembly)	Binary (Machine Code)
Programmer Focus	Data blocks (Tiles)	Individual threads/warps	Hardware instructions/scheduling
Ease of Use	Highest (automated management)	Moderate (manual management)	Hardest (raw assembly)
Control	Coarse-grained	Fine-grained	Absolute
Performance	Near-peak (for common ops)	Peak (hand-optimized)	Peak (extreme tuning)

Part 3:

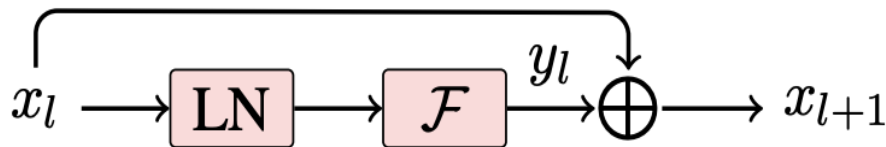
Hyperparameter

PreNorm vs PostNorm

Can be Either FFN or Attention Layers



(a) post-norm residual unit



(b) pre-norm residual unit

For **post-norm Transformers**, given a sub-layer l , the differential of $\mathcal{E}$ with respect to x_l can be computed by the chain rule:

$$\frac{\partial \mathcal{E}}{\partial x_l} = \frac{\partial \mathcal{E}}{\partial x_L} \times \prod_{k=l}^{L-1} \frac{\partial \text{LN}(y_k)}{\partial y_k} \times \prod_{k=l}^{L-1} \left(1 + \frac{\partial \mathcal{F}(x_k; \theta_k)}{\partial x_k} \right) \quad (5)$$

Where $\prod_{k=l}^{L-1} \frac{\partial \text{LN}(y_k)}{\partial y_k}$ represents the backward pass of the layer normalization, and $\prod_{k=l}^{L-1} \left(1 + \frac{\partial \mathcal{F}(x_k; \theta_k)}{\partial x_k} \right)$ represents the backward pass of the sub-layer with the residual connection.

Likewise, we have the gradient for **pre-norm**:

$$\frac{\partial \mathcal{E}}{\partial x_l} = \frac{\partial \mathcal{E}}{\partial x_L} \times \left(1 + \sum_{k=l}^{L-1} \frac{\partial \mathcal{F}(\text{LN}(x_k); \theta_k)}{\partial x_l} \right) \quad (6)$$

PreNorm vs PostNorm Comparison

- With the default baseline initialization, PostNorm fails to converge even with a long 16k-step warmup; using SMALLINIT fixes this instability. With the smaller initialization, the PostNorm model converges reliably across all warmup lengths.
- Table 5 shows PostNorm often fails without warmup for 5–6 layers, and even at 4 layers its performance is worse than PreNorm.

Xavier normal		# warmup steps		
		4k	8k	16k
Baseline	POSTNORM	fail	fail	5.76
	PRENORM	28.52	28.73	28.32
SMALLINIT	POSTNORM	28.17	28.20	28.62
	PRENORM	28.26	28.44	28.33

Table 2: Development BLEU on $en \rightarrow vi$ using Xavier normal initialization (baseline versus SMALLINIT).

	4 layers	5 layers	6 layers
POSTNORM	18.31	fails	fails
PRENORM	28.33	28.13	28.32

Table 5: Development BLEU on $en \rightarrow vi$ using NOWARMUP, as number of encoder/decoder layers increases.

Stability of Pre-LN vs. Post-LN Transformers

Traditional Post-LN Transformers require a warm-up stage because gradients near the output layer are excessively large at initialization, leading to instability. In Pre-LN Transformers (where layer normalization is inside the residual blocks), gradients are "well-behaved" and significantly smaller at initialization.

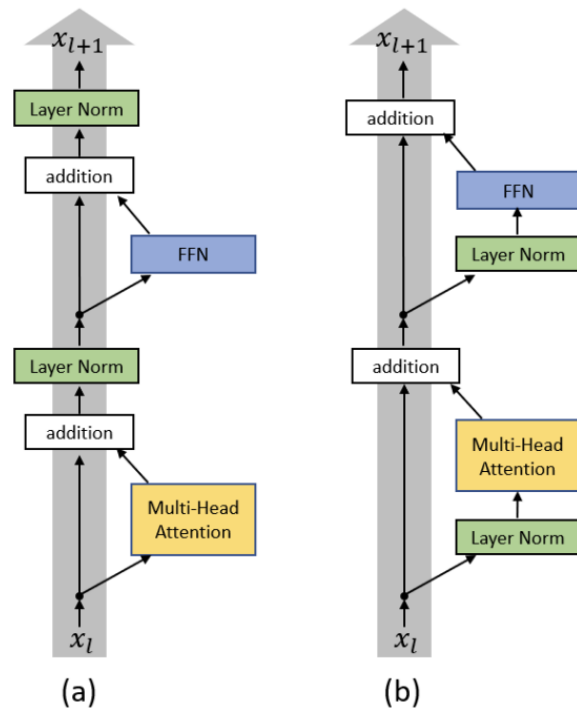
For the Post-LN Transformer with L layers, the gradient of the parameters of the last layer satisfies:

$$\left\| \frac{\partial \tilde{\mathcal{L}}}{\partial W^{2,L}} \right\|_F \leq \mathcal{O}(d\sqrt{\ln d})$$

For the Pre-LN Transformer with L layers:

$$\left\| \frac{\partial \tilde{\mathcal{L}}}{\partial W^{2,L}} \right\|_F \leq \mathcal{O}\left(d\sqrt{\frac{\ln d}{L}}\right)$$

We can see that for the Post-LN Transformer, the scale of the gradients to the last FFN layer is of order $\mathcal{O}(d\sqrt{\ln d})$, which is independent of L . For the Pre-LN Transformer, the scale of the gradients is much smaller.



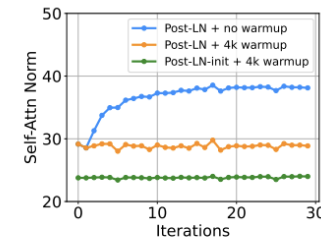
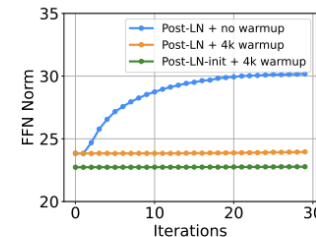
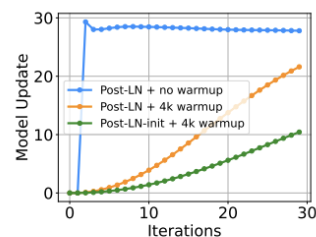
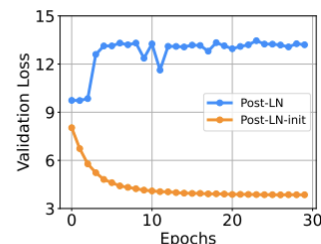
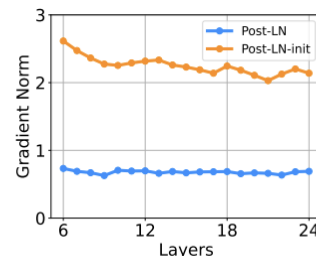
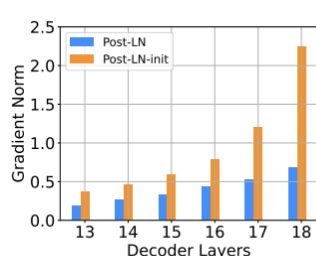
(a) Post-LN Transformer layer;
(b) Pre-LN Transformer layer

Better Initialization Stabilize Training Transformer w/ Post-LN

- Early large updates at the start of training push the model into a poor local optimum and amplify inputs to each layer normalization (LN). As training continues, gradients through LN shrink, causing severe gradient vanishing that prevents escape from the bad optimum and further destabilizes optimization.
- In contrast, Post-LN-init produces smaller, more stable updates and LN inputs, reducing gradient vanishing and yielding more stable optimization.

- The magnitude of gradient through LN is inversely proportional to the magnitude of its input $|x|$:

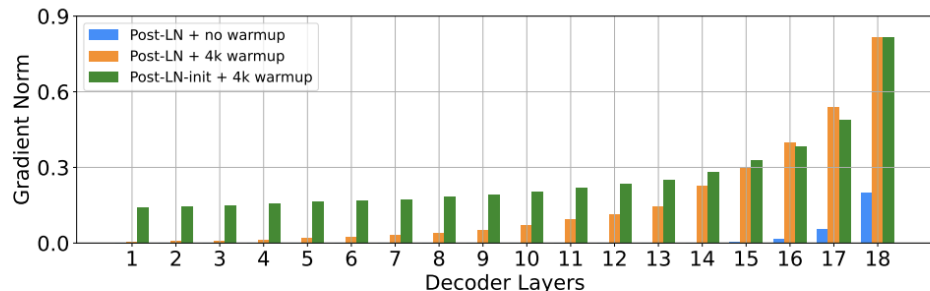
$$\left\| \frac{\partial \text{LN}(x)}{\partial x} \right\| = \mathcal{O}\left(\frac{\sqrt{d}}{\|x\|}\right)$$



(a) Accumulated model update

(b) Input from FFN to LN

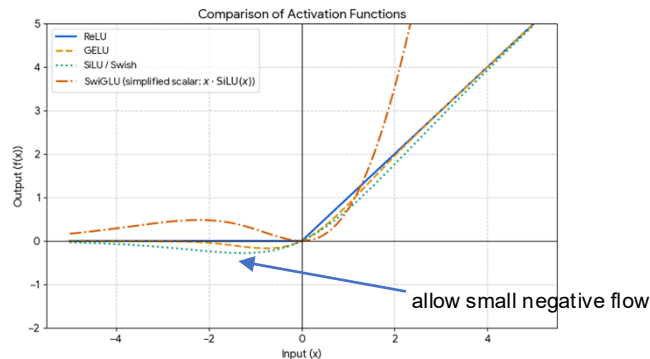
(c) Input from attention to LN



(d) Gradient norm in all decoder layers

Activation Functions in Language Models

- Modern Transformer architectures have largely shifted away from standard ReLU in favor of smoother, non-monotonic functions like GELU, SiLU, and SwiGLU.



Feature	ReLU	GELU	SiLU (Swish)	SwiGLU
Formula	$\max(0, x)$	$x \cdot \Phi(x)$	$x \cdot \sigma(x)$	$\text{Swish}(xW) \otimes (xV)$
Shape	Piecewise linear; hard zero for $x < 0$	Smooth curve; allows small negative values	Very similar to GELU; smooth and non-monotonic	Gated mechanism; requires two weight matrices
Gradients	Can suffer from "Dying ReLU" (zero gradient)	Gradients flow for small negative inputs	Smooth gradient flow across the origin	Highest complexity; most expressive gradients
Usage	Traditional models (BERT)	GPT-2, GPT-3, BERT	Early Llama, EfficientNet	Llama 2, Llama 3, PaLM

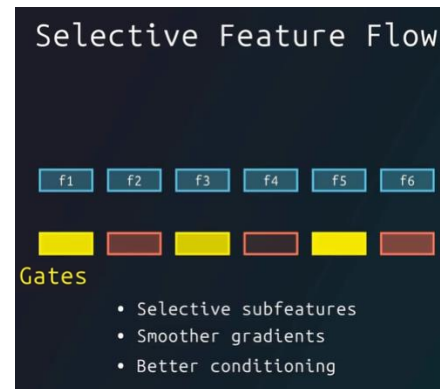
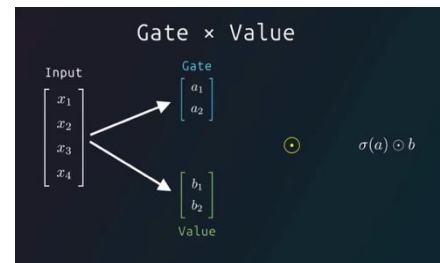
SwiGLU = SiLU Gate

- SwiGLU is a specialized architecture that incorporates the SiLU function as a gating mechanism to achieve superior performance, while SiLU is a standalone, non-linear activation function.

$$\text{SwiGLU}(x) = \text{SiLU}(xW_a) \odot (xW_b)$$

Components and Definitions:

- $\text{SiLU}(z) = z \cdot \sigma(z)$: This is the Sigmoid Linear Unit (also known as the Swish activation function), where $\sigma(z)$ is the sigmoid function.
- Gate Path** ($\text{SiLU } xW_a$): One linear transformation of the input x is passed through the SiLU activation to act as a "smooth gate."
- Value Path** (xW_b): A second linear transformation of the input x that provides the values to be gated.
- $\odot$ (**Element-wise $\times$**): The final output is the element-wise multiplication of the gated path and the value path.



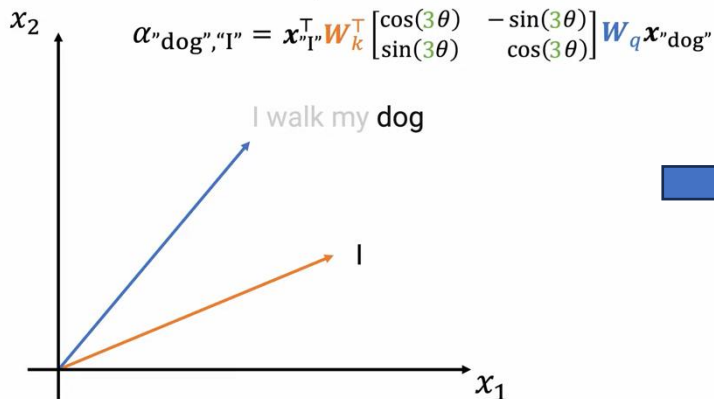
Rotary Positional Embeddings (RoPE)

$$q_{\text{"dog"}} = \begin{bmatrix} \cos(4\theta) & -\sin(4\theta) \\ \sin(4\theta) & \cos(4\theta) \end{bmatrix} W_q x_{\text{"dog"}}$$

Query vector for "dog"

$$k_{\text{"I"}} = \begin{bmatrix} \cos(1\theta) & -\sin(1\theta) \\ \sin(1\theta) & \cos(1\theta) \end{bmatrix} W_k x_{\text{"I"}}$$

Key vector for "I"



$$q_{\text{"dog"}} = \begin{bmatrix} \cos(6\theta) & -\sin(6\theta) \\ \sin(6\theta) & \cos(6\theta) \end{bmatrix} W_q x_{\text{"dog"}}$$

Query vector for "dog"

$$k_{\text{"I"}} = \begin{bmatrix} \cos(3\theta) & -\sin(3\theta) \\ \sin(3\theta) & \cos(3\theta) \end{bmatrix} W_k x_{\text{"I"}}$$

Key vector for "I"

